

Mini Sports Analytics Web App

Student Instruction Set · Herald College Kathmandu 2026

 Python 3.10+  Flask 2.3+  Matplotlib 3.7+  Seaborn 0.12+

 Status: Active

 License: MIT

1 Overview

The **Mini Sports Analytics Web App** is a Flask-based data analytics system designed for teaching the complete workflow of:

**Data → Cleaning → Analysis → Visualization →
Web Dashboard**

This project is developed for students at **Herald College, Kathmandu (2026)** under the **Centre for AI and Emerging Technologies**.

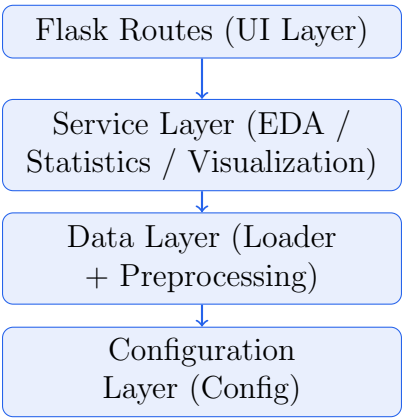
2 Learning Objectives

By working on this project, students will master:

Area	Skills
Project Structure	Organize real-world data analytics projects
Modular Design	Build reusable Python modules
Data Pipelines	Design end-to-end data processing flows
Exploratory Analysis	Generate insights using EDA and statistics
Visualization	Create plots with Matplotlib and Seaborn
Web Integration	Serve analytics through a Flask web application

3 Project Architecture

This project follows a **layered architecture design**:



Layer Responsibilities

Layer	Component	Responsibility
Presentation	Flask Routes	Handle HTTP requests, render templates
Business Logic	Service Layer	EDA, statistics, visualization generation
Data Access	Data Layer	Load and preprocess datasets
Configuration	Config	Manage paths, settings, environment variables

4 Directory Structure

```
mini-sports-analytics-web-app/
├── app/
│   ├── __init__.py           # Flask application factory
│   ├── routes.py             # Application routes (UI layer)
│   └── services/
│       ├── data_loader.py    # Load raw data
│       ├── preprocessing.py  # Clean and transform data
│       ├── eda.py            # Exploratory data analysis
│       ├── statistics.py     # Statistical computations
│       └── visualizations/
│           ├── helper.py     # Shared visualization utilities
│           ├── tournament.py # Tournament-specific plots
│           ├── geography.py  # Geographic visualizations
│           └── generate.py   # Main visualization orchestrator
├── data/
│   ├── raw/                  # Original datasets (DO NOT MODIFY)
│   └── processed/            # Cleaned datasets
├── static/
│   ├── images/
│   └── plots/                # Generated visualizations
└── templates/               # HTML templates for Flask
```

```
27     tests/
28         test_loader.py
29         test_preprocessing.py
30         test_statistics.py
31         test_visualizations.py
32
33     config.py           # Configuration settings
34     environment.yml     # Conda environment specification
35     requirements.txt    # pip requirements
36     README.md          # Project documentation
```

5 Application Design Pattern

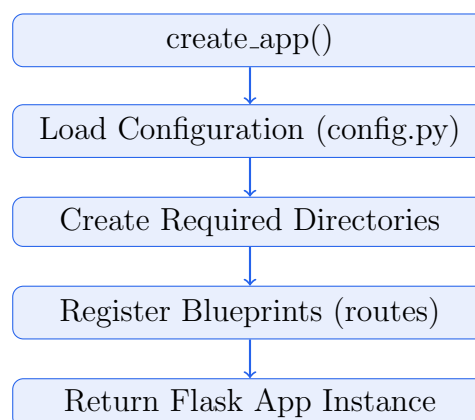
5.1 Flask Application Factory

This project uses the **Application Factory Pattern**—not a single run script.

```
1 def create_app():
2     """Flask application factory."""
3     app = Flask(__name__)
4     app.config.from_object('config.Config')
5
6     # Create required directories
7     os.makedirs('data/raw', exist_ok=True)
8     os.makedirs('data/processed', exist_ok=True)
9     os.makedirs('static/images/plots', exist_ok=True)
10
11    # Register blueprints
12    from app.routes import main_bp
13    app.register_blueprint(main_bp)
14
15    return app
```

Listing 1: app/_init_.py

5.2 Application Flow



6 Setup Instructions

6.1 1. Clone Repository

```
1 git clone <https://github.com/girisiman/mini-sports-analytics-web-app.git>
2 cd mini-sports-analytics-web-app
```

6.2 2. Create Environment

Option A: Conda (Recommended)

```
1 conda env create -f environment.yml
2 conda activate sports-analytics
```

Option B: pip

```
1 python -m venv venv
2 source venv/bin/activate # On Windows: venv\Scripts\activate
3 pip install -r requirements.txt
```

Warning

Use **ONLY ONE** environment system. Do **NOT** mix Conda and pip installations.

6.3 3. Run the Application

Windows (PowerShell)

```
1 $env:FLASK_APP="app:create_app"
2 flask run
```

Mac/Linux

```
1 export FLASK_APP=app:create_app
2 flask run
```

6.4 4. Open Application

<http://127.0.0.1:5000>

7 Dataset Guidelines

7.1 Important Rule

! DO NOT use pre-existing datasets blindly

Students are expected to **curate or create their own dataset**.

7.2 Recommended Data Sources

Sport	Recommended Data Sources
 Football	FIFA World Cup, UEFA Champions League, EPL, La Liga, NPL
 Cricket	IPL, ICC World Cup, T20 World Cup
 Basketball	NBA
 Other Sports	Olympics, Any sports tournament

7.3 Minimum Dataset Schema

Your dataset MUST include the following columns:

Column	Type	Description
year	Integer	Year of the match
date	Date	Match date (YYYY-MM-DD)
stage	String	Tournament stage
stadium	String	Stadium name
city	String	Host city
home_team	String	Home team name
away_team	String	Away team name
home_score	Integer	Goals scored by home team
away_score	Integer	Goals scored by away team
tournament	String	Tournament name

7.4 Example Dataset (CSV Format)

```

1 year,date,stage,stadium,city,home_team,away_team,home_score,away_score,tournament
2 2022,2022-11-20,Group,Ahmad Bin Ali Stadium,Al Rayyan,Qatar,Ecuador,0,2,FIFA World Cup
3 2022,2022-11-21,Group,Khalifa International Stadium,Doha,England,Iran,6,2,FIFA World Cup

```

8 Testing Instructions

Each module can be tested independently:

Module	Command
Data Layer	<code>python -m tests.test_loader</code>
Preprocessing Layer	<code>python -m tests.test_preprocessing</code>
Statistics Module	<code>python -m tests.test_statistics</code>
Visualization Module	<code>python -m tests.test_visualizations</code>

9 Visualization Output

All generated plots are stored in:

`static/images/plots/`

Each visualization function returns:

Return Value	Description
<code>title</code>	Human-readable plot title
<code>filename</code>	Generated filename (e.g., <code>goals_distribution.png</code>)
<code>caption</code>	Descriptive caption for UI display

10 Module Responsibilities

10.1 1. Data Layer (`services/data_loader.py`)

```
1 def load_raw_data():
2     """Load raw dataset from data/raw/"""
3     pass
4
5 def get_file_paths():
6     """Manage file paths for raw and processed data"""
7     pass
```

10.2 2. Preprocessing Layer (`services/preprocessing.py`)

```
1 def clean_data(df):
2     """Clean dataset: handle missing values, fix data types"""
3     pass
4
5 def save_processed_data(df):
6     """Save cleaned data to data/processed/"""
7     pass
```

10.3 3. Analysis Layer

EDA (services/eda.py)

```
1 def generate_summary(df):
2     """Generate descriptive statistics and data overview"""
3     pass
4
5 def detect_outliers(df):
6     """Identify outliers in numerical columns"""
7     pass
```

Statistics (services/statistics.py)

```
1 def compute_team_stats(df):
2     """Calculate team performance metrics"""
3     pass
4
5 def compute_tournament_stats(df):
6     """Calculate tournament-level statistics"""
7     pass
```

10.4 4. Visualization Layer

Helper (visualizations/helper.py)

```
1 def save_figure(fig, filename):
2     """Save matplotlib figure to static/images/plots/"""
3     pass
4
5 def get_plot_metadata(title, filename, caption):
6     """Return standardized plot metadata"""
7     pass
```

Tournament (visualizations/tournament.py)

```
1 def plot_goals_distribution(df):
2     """Create goals distribution histogram"""
3     pass
4
5 def plot_top_scorers(df):
6     """Create bar chart of top scoring teams"""
7     pass
```

Geography (visualizations/geography.py)

```
1 def plot_stadium_distribution(df):
2     """Create pie chart of matches per stadium"""
3     pass
4
5 def plot_heatmap_by_city(df):
6     """Create heatmap of matches by city"""
7     pass
```

10.5 5. Integration Layer (visualizations/generate.py)

```
1 def generate_all_plots(df):
2     """Orchestrate all visualization generation."""
3     plots = []
4     plots.append(plot_goals_distribution(df))
5     plots.append(plot_stadium_distribution(df))
6     # ... call all plot functions
7     return plots
```

10.6 6. Routes Layer (app/routes.py)

```
1 @main_bp.route('/')
2 def index():
3     """Render main dashboard."""
4     # Call service layer functions
5     # Pass data to template
6     return render_template('index.html', **context)
```

Warning

No heavy computation inside routes! Keep routes thin—delegate to service layer.

11 Expected Learning Outcomes

After completing this project, students will understand:

Data Engineering

- ✓ Pipeline design and implementation
- ✓ Data cleaning strategies
- ✓ File I/O and path management
- ✓ Modular code organization

Data Analysis

- ✓ Exploratory Data Analysis (EDA) techniques
- ✓ Statistical summarization
- ✓ Pattern identification
- ✓ Insight generation

Visualization

- ✓ Storytelling with data
- ✓ Static plotting with Matplotlib/Seaborn
- ✓ Chart type selection
- ✓ Publication-quality figures

Web Integration

- ✓ Flask routing and blueprints
- ✓ Template rendering with Jinja2
- ✓ Modular backend design
- ✓ Application factory pattern

Software Development

- ✓ Version control (Git)
- ✓ Environment management (Conda/pip)
- ✓ Testing strategies
- ✓ Documentation practices

12 Troubleshooting

Common Issues

Issue	Solution
ModuleNotFoundError	Activate virtual environment: <code>conda activate sports-analytics</code>
Port 5000 already in use	Use <code>flask run --port=5001</code>
Plots not showing	Check <code>static/images/plots/</code> directory permissions
Data loading fails	Verify CSV format and file path in <code>config.py</code>

Getting Help

1. Check the test files for expected function signatures
2. Review the directory structure for correct file placement
3. Verify environment activation with `conda info --envs`
4. Consult the official Flask and Matplotlib documentation

13 Resources

Documentation

- [Flask Documentation](#)
- [Pandas Documentation](#)
- [Matplotlib Documentation](#)
- [Seaborn Documentation](#)

Data Sources

- [Kaggle Sports Datasets](#)
- [ESPN Stats](#)
- [Sports Reference](#)

14 Final Note

Final Note

This project simulates a **real-world analytics system**. Students are expected to think like:

- **Data Engineers** — Building robust data pipelines
- **Data Analysts** — Extracting meaningful insights
- **Software Developers** — Creating maintainable code
- **UI/UX Designers** — Presenting data effectively

Good luck, and happy coding!